

Audio Classification and Music Recommendation

Aren Vista Hailey Davio
University of Maryland, Baltimore County
{arenv1, hdaviol}@umbc.edu

Abstract

From Apple Music to Spotify, song suggestion algorithms have become a hallmark of music streaming platforms. Many of these platforms base music recommendations on surface-level factors such as genre or artist, but taking a deeper look at audio data suggests that there may be a way to provide users with a more personalized experience. This project aims to take a closer look at what makes audio files unique, utilizing visual information via spectrograms and sentiment encoding via lyrics. By using a multimodal system and training our model with these factors in addition to song metadata, we aimed to create a more accurate and refined song recommendation system based on existing user preferences.

A link to the code can be found hosted on GitHub at <https://github.com/arenvista/AudioRecomendation>.

1. Introduction

In order to maximize user retention, modern day music streaming services prioritize effective but lightweight algorithms to determine which songs get recommended to which users. Some rely on cursory music metadata: if a user likes a song from a particular artist, for example, the algorithm might suggest another song from that same artist. Many platforms also base their suggestions on the listening habits of other similar users: if two users have similar song histories, the platform may recommend their songs to each other. These approaches do not always guarantee that the suggested song matches the rest of a user's preference in areas like genre or tone. Moreover, they may rely heavily on other users' histories and patterns. In contrast, we aimed to design a more independent recommendation process that could perform well even without using other users' behaviors to contextualize an individual user's preferences.

When it comes to evaluating the songs themselves, large platforms generally employ a more simplistic audio analysis in order to keep their application lightweight and streamline the user experience. This simplicity, however, often comes at the price of high feature-based accuracy. The goal of our

project was to take a more in-depth look at what defines music and how that definition relates to listening preferences. We examined other data channels and interpretations in order to create a comprehensive, multi-dimensional approach to evaluating music tracks.

By shifting the focus from a song's artist or category to what it actually sounds like, this method bridges the gap between audio characteristics and user preferences. This approach not only results in highly accurate recommendations but also highlights lesser-known artists that may otherwise be buried by traditional, popularity-driven algorithms.

To achieve this, this system focuses on three objectives:

1. *Spectrogram Representation:* The system will convert each .mp3 file into a spectrogram. This allows the audio data to be treated as image data, creating a visual representation of a track's frequencies and amplitudes over time. By treating audio as an image, the system can utilize deep learning models (such as Convolutional Neural Networks) to extract complex audio features like timbre, tempo, and structure.
2. *User Profiles:* This approach creates a series of fake "users". Each has their own invented listening history, a series of songs that define their music taste. These fake users are independent of an actual music platform, meaning rather than clicked genres or liked artists affecting recommendations, it's the actual audio qualities of a user's song history. This creates a stronger approximation of user preferences based primarily on their music tastes.
3. *Generating Content-Based Recommendations:* Using each user's profile, the model will compare the user's listening history against a track database. By assessing the signatures of songs from the dataset and the user, the engine will suggest new songs that fit the user's established tastes.

Data Collection Labeled audio tracks will be sourced from FMA: A Dataset For Music Analysis [?]. The audio data is MP3-encoded. Each song will be converted from .mp3 format to a spectrogram to be processed as visual data. Other audio data provided by the dataset (volume, song length, etc.) may also play a role in data processing.

Metadata Similar to existing music streaming platforms, the model takes into account each track’s metadata. The genre is a simple way to get the general impression of a track. Trends and grouping may be gleaned from information about the artist and album. Tags provide supplementary information. The number of listens provides insight into whether popular songs have more in common with each other than their unpopular counterparts. These data points, alongside many others contained in the dataset, provide a strong baseline for song analysis.

Lyrics While metadata can provide general details about a song, lyrics provide more insight into semantic meaning and tone. This was found to be especially useful because these aspects are rarely captured by other song recommendation algorithms. Though this approach came with its own set of limitations, it generally gave the model a fuller picture of each track.

2. Related Work

2.1. AST: Audio Spectrogram Transformer

The Audio Spectrogram Transformer (AST) that we implemented for interpreting audio data as spectrogram images is a purely attention-based model used for audio classification. It was conceptualized as a way to test whether the reliance on a CNN was necessary in the domain of audio classification, and how domain tasks could be accomplished with a convolution-free model. Historically, CNN-attention hybrids have been used to meet similar goals. It was common to see CNN with self-attention mechanisms layered on top excelling at audio classification and speech & emotion recognition. The AST mirrors the Vision Transformer (ViT) for vision tasks, using a simpler Transformer architecture with fewer parameters. Notably, while the AST’s input lengths ranged from 1 to 10 seconds, our inputs were consistently 30 seconds. While audio tasks often suffer from a shortage of available data compared to image tasks, AST was able to combat that by using a modified, pretrained Vision Transformer (ViT). This modified version condensed the 3-channel input of ViT into a single channel for AST and adapted positional embeddings for variable-length input, resulting in a much quicker training process when compared to randomized initialization [?].

2.2. Music Recommendation on Spotify using Deep Learning

When outlining the goals of our project, we used existing music platforms as a baseline for comparing song recommendation methods and results. While Spotify, one of our primary inspirations, has no public documentation on the algorithm it uses for music recommendation, its approach has been speculated to include multiple machine learning algorithms. One of their major algorithm components is Con-

tent Filtering, which involves recording and analyzing the habits of existing listeners. Using these learned habits, the platform can more reliably predict the behaviors and listening habits of similar users. This filtering uses two methods: one is an “Exploitative” method that suggests songs based on existing data, preferences, and habits of similar users; another is an “Explorative” method based more heavily on track features which suggests songs that contrast existing preferences. Natural Language Processing is likely another facet of Spotify’s algorithm, wherein text from song titles, album titles, tags, and searches are used to predict user preferences. It is believed that Audio Models are also used to examine the audial properties of songs, incorporating that information into recommendations. A combination of these approaches blend to create an effective song system with high user retention, highlighting both potential parallels and contrasts for our project [?].

2.3. Free Music Archive

The Free Music Archive (FMA) dataset that we used for training our model was derived from an open library of the same name. Curated by WFMU, a long-running U.S. radio station, this dataset highlights legally accessible song tracks and has established itself as a large-scale, high-quality library for audio-based projects. It sets itself apart from many other music datasets, such as OMRAS2 and Codaich, by containing the actual audio files. Previously, the standard dataset for Music Information Retrieval (MIR) was GTZAN, which contained only 1000 clips from 10 genres. In contrast, FMA contains over 100,000 clips from over 160 genres. It also contains a wealth of metadata, with metrics from the actual songs in addition to user interactions. It works especially well for Music Genre Retrieval (MGR) due to its feature organization. Features include several sub-genres as well as a built-in genre hierarchy that follows original track annotations. A collection of over 500 pre-computed features also provide valuable insight into track identification. A combination of these attributes make the dataset exceptionally useful for a variety of tasks with the Music domain [?].

2.4. Learning Transferable Visual Models From Natural Language Supervision

Contrastive Language-Image Pretraining (CLIP) is a model designed to align image and text embeddings in a purely attention-based format. Its structure inspired the idea to align our audio-based visual representations with other input like lyrics. Its effectiveness can, in part, be attributed to its massive dataset. The development team cultivated a new dataset of 400 million pairs of images and text, forming a diverse collection of information and greatly reducing the risk of over-fitting. Pretraining approaches with both ResNet-50 and a slightly modified version of the Vi-

sion Transformer (ViT) were both used for experimentation. The text encoder is a Transformer with architectural modifications. A decayed learning rate, Adam optimizer, and decoupled weight decay regularization were all used during training. Initial hyperparameters were based on a combination of grid & random searches with manual tuning. The model was ultimately trained to predict text and image pairings, enabling flexible zero-shot transfer that outperformed many comparable approaches like ViR-M and SimCLRv2. The team found success with engineering the text associated with each image to be more specific and context-focused. The model also performed well when it came to representation learning. Though the performance of smaller CLIP models was more reliant on their training dataset, the models scale well and are able to learn a broader spectrum of tasks. A combination of custom architecture and training ultimately made CLIP a robust model that excelled even when compared to human performance [?].

2.5. CLAP: Learning Audio Concepts From Natural Language Supervision

Another model prevalent in the MIR space is the audio alternative to CLIP, CLAP (Contrastive Language-Audio Pre-training). We only discovered CLAP towards the end of the project’s development and were unable to implement it, but it pairs well with our project goals by introducing a clean method of directly aligning audio with text. It aims to emulate the way that humans are able to recognize and contextualize audial input in the same way that CLIP does with images. It has been trained to work with multiple domains, from Sound Event Classification to Speech-related tasks. Their model also had applications in our domain, addressing Music tasks like genres, strokes, and tonics. It employs Zero-Shot predictions, avoiding the limitations of training with class labels and strict predictions at inference time. CLAP utilizes two encoders and contrastive learning in order to bring text and audio data into a multimodal space, essentially combining the manual chaining of our ViT and CLIP embeddings. The two encoders process audio and text, learning the embedding space based on the inputs’ similarities and differences. Once learned, CLAP is capable of flexible class predictions and a variety of downstream tasks [?].

3. Methods

Our methodology consisted of four main stages: dataset preparation, audio representation with the Audio Spectrogram Transformer (AST), multimodal alignment with CLIP, and dimensionality reduction with principal component analysis (PCA).

3.1. Dataset

We primarily used the Free Music Archive (FMA) dataset [?]. FMA contains 106,574 tracks from 16,341 artists and 14,854 albums, spanning 161 genres. Training on such a diverse collection encouraged the model to generalize more effectively and produce more accurate embeddings for previously unheard audio.

3.2. Audio Representation with AST

Our training approach began with using a pre-trained version of the Audio Spectrogram Transformer (AST) developed by Gong et al [?]. This transformer first takes each .mp3 file from our dataset and converts it into a spectrogram. A spectrogram is a visual representation of audio, depicting the frequency and amplitude of the songs over time. It shows a greater level of detail than comparable visual representations such as waveforms, making it ideal for analyzing audio tracks in more depth [?].

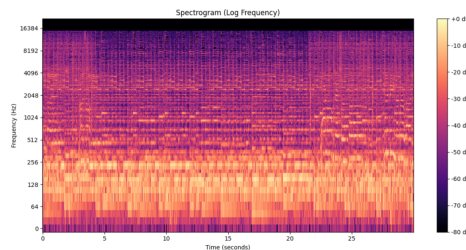


Figure 1. An example spectrogram, similar to those generated by AST. It is a visual representation of one of our dataset tracks, “Calling For You” by The Twin Atlas.

Only using 30 second clips meant that every spectrogram shared the same scale and axes, resulting in more uniform training data. These smaller clip sizes were easier to process, but did not make full use of the AST’s ability to handle variable-length inputs. Overly-consistent training data may have also negatively affected the model’s ability to generalize, potentially factoring into reduced performance on testing data that did not fit the 30 second structure.

The AST divides each spectrogram into overlapping 16×16 image patches before linearly projecting them into one-dimensional patch embeddings. These patch embeddings are added with learnable positional embeddings and prepended with a classification token. The output embedding becomes the Transformer input, while the classification token is used with a linear layer for classification purposes [?]. Using this approach provided a valuable way to process audio in a visual format.

Notably, the AST was fine-tuned on AudioSet, a labeled dataset cultivated by Google that contains over two million audio clips. The items within AudioSet are sound events derived from YouTube, meaning that the model’s pre-trained

weights required fine-tuning in order to better suit the music domain [?].

3.3. Multimodal Alignment with CLIP

The second major component of our multimodal approach was the Contrastive Language-Image Pretraining (CLIP) vision transformer. While AST handled the audio-based visual component of our methodology, CLIP’s text processing allowed us to incorporate text input and semantic meaning. For every song that contained vocals, we generated a respective set of lyrics. These lyrics were passed as input to the CLIP model, which then produced a 512-dimension lyric vector. The 768-dimension audio vector produced by the AST was compressed to 512 dimensions in order to match the lyric vector’s dimensions [?].

From there, the symmetric cross-entropy loss was computed and used to more closely align each audio vector with its respective lyric vector. We followed a similar approach to Radford et al., using a temperature to allow the model to be flexible with the way it handles distances and forms clusters [?]. Once loss is computed, backpropagation passes the value back through the network to determine the gradient and reduce the loss in the next iteration. Following standard protocol for Transformer models, AdamW was used to update the model weights using a combination of momentum and scaling with added weight decay [?].

3.4. Principal Component Analysis

The resulting 512-dimensional output was centered and condensed to prime the embeddings for principal component analysis. The axes, PC1, PC2, and PC3, are calculated in order to show the greatest variance between data points. Once the axes are determined, all resulting vectors are projected onto the PCA space. As a linear approach, PCA gave us a more interpretable output while reducing the risk of imaginary clusters sometimes seen with UMAP or t-SNE [?] [?].

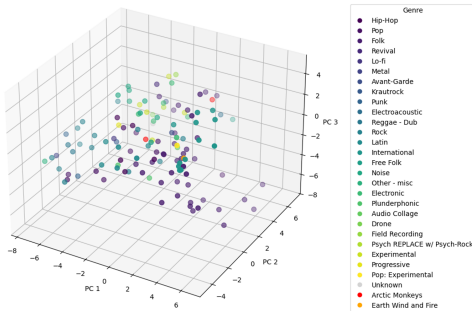


Figure 2. The PCA projection of a small subset of data with genres distinguished by color.

4. Experiments

Prior to completing any training, we evaluated our approach using the pre-trained versions of AST and CLIP and a small subset of our data. Even without any fine-tuning, the model showed early promise of grouping songs. Solely based on the PCA interpretation, the distance between two instrumental piano songs (“Here With You” and “Piano 2” by Askew Ed) was 2.0168. The distance between “Here With You” and a heavy metal song, “The White Mice” by White Mice, was 13.7109. This gave us early confirmation that even without trained weights, our model was able to closely group similar songs while pushing dissimilar songs further apart.

In order to quickly validate the ground-truth, we first took a look at the genres of the songs being recommended for each user. If the recommended songs were within the same genre as those already within the fake user’s library, it was a promising early sign that the clustering was working as expected.

We also evaluated the distance between songs included in each user’s library. Each of the five songs were selected to be highly similar to one another, so we were able to verify cluster effectiveness by measuring the intra-cluster distance between the songs. Metal and rock songs, such as those used for Users 02 and 05, contained more variety in characteristics like tempo and volume and therefore exhibited large intra-cluster distances. In contrast, reggae songs such as those used for User 01 had more similarities in their pacing, instrumentals, and vocals and were therefore clustered closer together.

User_ID	Genre	Intra-Cluster Distance
01	Reggae	2.93879742
02	Rock/Metal	7.0193084
03	70s/Psychedelic	5.6717442
04	Classical	4.7008497
05	Metal	7.0248702
06	Jazz/Bebop	5.6619262
07	Pop-Punk	5.0302312
08	Bossa/Jazz	6.1153690
09	2010s Pop	4.4693859
10	Club	3.7002654
11	Alt Indie	3.57691073
12	Alt Hip-Hop	4.7994033

Table 1. Intra-cluster distances across user profiles and their respective genres

We also evaluated inter-cluster measurements, comparing the distances between each user’s cluster. When curating users, we made sure to create fake song libraries that spanned a variety of genres, from instrumental classical mu-

sis to lyric-heavy rock songs. The distance between user clusters of different genres was another indicator of how well the songs were clustering. We found that classical music tended to be furthest-removed from other songs. This suits our hypothesis, as classical music is set apart by its lack of lyrics and distinct instrumental characteristics. Conversely, the danceable nature and catchy lyrics of 2010s pop and alt hip-hop meant that the two genres were relatively close to each other. There were similar trends across genres, but particularly relevant relationships are highlighted in Table 2.

Genre 1	Genre 2	Inter-Cluster Distance
Bossa/Jazz	Club	9.2047650
Classical	Alt Hip-Hop	11.8344327
Classical	Reggae	12.6708038
Classical	Club	11.1681592
Rock/Metal	Pop-Punk	5.5034570
2010s Pop	Alt Hip-Hop	4.596454
Jazz/Bebop	Alt Indie	5.15431743

Table 2. Inter-cluster distances between clusters of different genres

5. Conclusion

This project demonstrated the potential of combining audio and lyrical information for music understanding and recommendation, while also highlighting several practical and conceptual limitations. In particular, inconsistent alignment between lyrical meaning and musical character, the manual construction of user profiles, infrastructure-related obstacles, and incomplete lyric coverage all constrained the scope of the evaluation.

Given more time, a useful extension would be to automate preprocessing by writing a script that segments longer tracks into 30-second clips and averages the resulting predictions. Another promising direction would be to develop a representation that captures the semantic content conveyed by musical notes themselves. For example, one could draw on ideas from music theory to group chords, motifs, or harmonic progressions and model them as a “corpus” analogous to natural language, thereby enabling analysis even for tracks without lyrics.

6. Limitations

Language often contradicts a song’s overall tone. Incorporating the semantic content of lyrics introduced a challenge: meaning conveyed by lyrics does not always align with the song’s other characteristics. For example, an up-beat pop track may feature melancholic or introspective lyrics, producing a semantic representation that appears inconsistent with the song’s genre, mood, or instrumentation.

Such mismatches can weaken the effectiveness of the contrastive learning approach [?] [?]. In particular, when lyrical semantics conflict with the broader musical context, the resulting embeddings may capture contradictory signals, making it more difficult for the model to learn coherent cross-modal relationships.

User data was difficult to obtain. There was no straightforward way to query popular music platforms such as Spotify [?] for their top playlists. To evaluate the song recommendation algorithm more realistically, the sample user profiles were therefore compiled by hand.

For each “user,” a set of songs within the same genre, or at least similar enough to approximate a coherent listening preference, was selected. This introduces a certain level of error, as hand-selected songs will not always be precisely within the same genre or tone. Moreover, the model was trained on 30-second audio clips from the FMA dataset [?], so each selected song also had to be trimmed to a 30-second excerpt to ensure the best possible compatibility with the training data.

Large dataset download issues. Downloading a large dataset proved unreliable, as the `wget -c` command repeatedly stalled at 0-byte transfers. After ruling out problems with the command itself, it was determined that the issue was likely related to the selected partition. Subsequent correspondence with HPC staff helped diagnose the problem and identify a workable solution.

Missing FFmpeg dependency for multicore jobs. Training required `libcodecs.dll` from FFmpeg. Although this dependency was available on the single-core cluster, the necessary module was not available for multicore jobs, as confirmed by checking the output of module avail. To work around this limitation, a static build was downloaded from the Git repository and the executable was added to the `PATH` environment variable.

Acquiring a lyrics dataset was challenging. A centralized repository containing lyrics for all tracks in the collection could not be found. In addition, some tracks, such as instrumentals, do not contain lyrics at all, which introduced an additional design consideration when constructing the dataset.